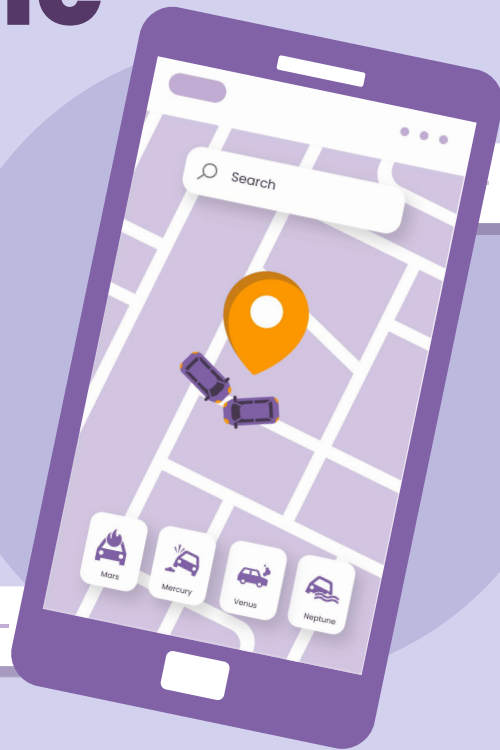


Converting Traffic Camera Data to Real-World Coordinates



Problem:

Pixel data from traffic cameras has to automatically be converted into real GPS locations so emergency services can be dispatched immediately, without any manual setup per camera and without failing in adverse weather conditions like snow or darkness.

**Hundreds of
Cameras**

**Adverse
Weather**

**No Manual
Calibration**

Approaches

Satellite Image Matching for Pixel-to-GPS Homography	Use a mathematical matrix computed from matched points between the camera feed and satellite imagery to instantly convert any pixel coordinate to a GPS coordinate.
Vanishing Point Detection for Automatic Camera Calibration and Pixel Reverse-Projection	Track vehicle trajectories in the pixel data to find where they converge – the vanishing point – which mathematically reveals the camera's orientation, focal length, and tilt, and when combined with the camera's known GPS location, allows any pixel coordinate to be reverse-projected from image space into a real-world GPS coordinate.
3D Vehicle Bounding Box Alignment for GPS Localization	Detect vehicles in the pixel data, classify their type, and compare their known real-world dimensions against how they appear distorted in the image, whereafter their exact distance and angle from the camera can be mathematically calculated and converted into a GPS coordinate.

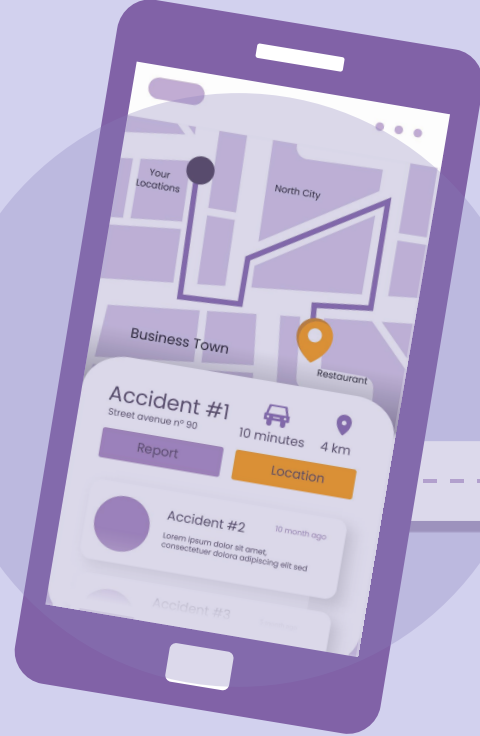


1. Satellite Image Matching for Pixel-to-GPS Homography

The core idea is to compute a mathematical matrix – called a Homography Matrix H – that permanently maps every pixel coordinate in a camera's frame to a real-world GPS coordinate using anchor points. Once H is computed for a given camera, converting any pixel (u, v) to a GPS coordinate (latitude, longitude) is a single matrix multiplication.

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

A satellite image containing geospatial metadata (GeoTiff file) is generated using the camera location. After, 8-12 anchor points (visual patterns in both camera feed and satellite image) are created with **IoFTR** and are passed into `cv2.findHomography()`, which returns the H matrix. This gets multiplied by the pixel-coordinate matrix in order to find the real-world coordinates with `cv2.perspectiveTransform()`.



- Fleischer, J.P., et al. (2025). Measuring Braking Behavior Using Vehicle Tracking and Camera-to-Satellite Homography Rectification. arXiv:2601.17558. <https://arxiv.org/abs/2601.17558>
- Milosavljević, A., et al. (2017). A Method for Estimating Surveillance Video Georeferences. ResearchGate Publication 319242972. <https://www.researchgate.net/publication/319242972>



Vanishing Point Detection for Automatic Camera Calibration and Pixel Reverse-Projection



When a camera looks down a straight road, parallel lane lines converge toward a single point on the horizon in the image. That point is called the vanishing point, and can be better estimated using vehicle trajectories. Its pixel location in the image mathematically encodes the camera's complete orientation, meaning if we know the vanishing point, we know its tilt, rotation, and focal length, which is everything we need to reverse-project any pixel coordinate into a real-world GPS coordinate. This can be achieved with matrix calculations ($P = K * [R \mid T]$).

- Kocur, V., & Ftáčnik, M. (2021). Traffic Camera Calibration via Vehicle Vanishing Point Detection. ResearchGate Publication 354499458. <https://www.researchgate.net/publication/354499458>
- Han, C., Liu, Z., & Zhang, Y. (2024). Benchmark Dataset and Deep Model for Monocular Camera Calibration from Single Highway Images. PubMed Central PMC12473645. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12473645>



Reverse-Projection Calculations

By tracking vehicle trajectories in the pixel data and finding where they converge, we get two vanishing points – one in the road direction (VP1) and one perpendicular to it (VP2). From these two points, focal length is derived first by drawing vectors from the image center to each vanishing point, taking their dot product, negating it, and taking the square root. Pitch (tilt) is then calculated by measuring how far VP1 sits above or below the image center and dividing by focal length inside an arctangent – the further VP1 is from center vertically, the more the camera is tilted. Yaw (rotation) is calculated the same way but horizontally – how far VP1 sits left or right of center tells us which direction the camera is facing. Roll is derived from VP2's position relative to the image center. All four values – focal length, pitch, yaw, and roll – come entirely from the pixel positions of the two vanishing points and require no external measurements.

$$P = K \times [R \mid t]$$



Once focal length, pitch, yaw, and roll are known, they are assembled into two matrices – the intrinsic matrix K encoding the camera's internal optics, and the rotation matrix R encoding its orientation in 3D space. For any pixel coordinate (u, v) where an incident is detected, a ray is cast backward through the camera by multiplying the inverse of K by the pixel coordinate, producing a direction vector pointing from the camera into the real world. That ray is then intersected with the ground plane – since the road is flat, Z equals zero, and the camera's known mounting height gives us the exact point where the ray hits the ground, producing a metric X and Y coordinate relative to the camera. Finally, since Iowa DOT metadata gives us the camera's GPS location, the metric X and Y offsets are converted to latitude and longitude by dividing by the number of meters per degree at that location – approximately 111,320 meters per degree latitude and 111,320 times the cosine of the latitude per degree longitude – yielding the final real-world GPS coordinate of that pixel.



3D Vehicle Bounding Box Alignment for GPS Localization

Since vehicle dimensions are legally standardized by the DOT, every detected vehicle is essentially a known-size object appearing in the image. After measuring how that known-size object appears distorted by perspective in the pixel data, the system mathematically calculates exactly how far away it is and at what angle – and converts that into a GPS coordinate relative to the camera's known location.

Once the deep learning model detects a vehicle and classifies its type, we know its real physical dimensions from DOT standards. By measuring how large that vehicle appears in the pixel data and dividing the known real size by the apparent pixel size scaled by the focal length, we get the exact distance from the camera to the vehicle in meters. The bearing angle is then derived from how far the vehicle's center pixel sits left or right of the image center, telling us the direction to the vehicle relative to where the camera is pointing. With distance and bearing known, we project them onto the ground plane using trigonometry. Those metric offsets are then converted to degrees of latitude and longitude by dividing by the number of meters per degree at Iowa's latitude – approximately 111,320 meters per degree – and added to the camera's known GPS coordinates, producing the final real-world latitude and longitude of the detected vehicle.

- Sochor, J., Juránek, R., & Herout, A. (2017). Traffic Surveillance Camera Calibration by 3D Model Bounding Box Alignment for Accurate Vehicle Speed Measurement. ResearchGate Publication 313879158.
<https://www.researchgate.net/publication/313879158>
- Tang, X., Song, H., Wang, W., & Zhao, C. (2022). CenterLoc3D: Monocular 3D Vehicle Localization Network for Roadside Surveillance Cameras. ResearchGate Publication 359517817.
<https://www.researchgate.net/publication/359517817>

